



Department of Electrical & Electronic Engineering
Rajshahi University of Engineering & Technology

LAB Report

Course Code : **EEE-3100**

Course Title : **Electronics Shop Practice**

Experiment No. : 06

Experiment Name : Smart Line Following and Obstacle Avoidance Robot with Directional Feedback and Movement Tracking.

Submitted By:

Name : Labib Marwan Hoque

Roll : 2101064

Section : B

Submitted To:

Asif Zaman Rizve

Lecturer

Department of Electrical & Electronic
Engineering

Rajshahi University of Engineering & Technology

Date of Experiment : 12 May, 2025

Date of Submission : 26 May, 2025

Experiment No.: 06

Experiment Name: Smart Line Following and Obstacle Avoidance Robot with Directional Feedback and Movement Tracking.

Objective:

- Line Following: Using infrared (IR) sensors to detect and follow a predefined path.
- Obstacle Avoidance: Utilizing an ultrasonic sensor mounted on a servo motor to detect and navigate around obstacles.
- Directional Feedback and Movement Tracking: Implementing logic to determine optimal movement directions based on sensor inputs.

Theory:

A line follower robot with obstacle avoidance combines two key functionalities: following a line and avoiding obstacles. It uses IR sensors to detect a line on the ground and an ultrasonic sensor to detect obstacles in its path. The IR sensors help the robot stay on the line by adjusting its direction based on the sensor readings, while the ultrasonic sensor measures the distance to obstacles, prompting the robot to stop and avoid collisions.

A microcontroller (e.g., Arduino) processes sensor data and controls the motors using a motor driver. The robot continuously follows the line unless an obstacle is detected, at which point it pauses, maneuvers around the object, and returns to the line.

This robot is ideal for automation, autonomous navigation, and educational purposes, providing practical learning in sensor integration, control systems, and robotics. It's commonly used in industrial automation for transporting goods and educational projects to teach basic robotics concepts. Key challenges include sensor calibration and real-time decision-making for effective obstacle avoidance.

The robot is built around an Arduino UNO microcontroller, an L298N motor driver, IR sensors, an ultrasonic sensor, and a servo motor.

1.Line Following Mechanism

IR Sensors: Two IR sensors (left and right) are used to detect the presence of a line (typically black) on a contrasting surface (usually white). The sensors output digital signals indicating whether they detect the line beneath them.

Logic:

- Both Sensors on White: Move forward
- Left Sensor on Black, Right on White: Turn left
- Right Sensor on Black, Left on White: Turn right
- Both Sensors on Black: Stop or perform obstacle detection

This logic allows the robot to adjust its movement to stay aligned with the path.

2.Obstacle Detection and Avoidance

Ultrasonic Sensor: Measures the distance to objects in front of the robot. If an obstacle is detected within a certain threshold, the robot initiates avoidance manoeuvres.

Servo Motor: Rotates the ultrasonic sensor to scan the environment, typically to the left and right, to determine the best path around the obstacle.

Avoidance Logic:

- If an obstacle is detected ahead, the servo rotates the ultrasonic sensor to scan left and right.

- The robot compares distances on both sides and chooses the direction with more clearance.
- Executes a turn in the chosen direction and continues moving forward.

3.Motor Control with L298N Driver

L298N Motor Driver: Controls the speed and direction of two DC motors.

- Enable Pins (enA, enB): Control the speed of the motors using Pulse Width Modulation (PWM)
- Input Pins (in1, in2, in3, in4): Determine the direction of motor rotation

Motor Control Functions:

- Forward: Both motors rotate forward
- Turn Left: Right motor moves forward, left motor stops or moves backward
- Turn Right: Left motor moves forward, right motor stops or moves backward
- Stop: Both motors stop

4.Servo Control

ServoPulse Function: Generates precise pulses to control the angle of the servo motor, allowing the ultrasonic sensor to scan different directions.

5.Distance Measurement

Ultrasonic_read Function: Triggers the ultrasonic sensor and calculates the distance to an object based on the time taken for the echo to return.

6.Decision-Making Functions

Check_side Function: Determines the distance on either side of the robot by rotating the servo-mounted ultrasonic sensor.

compareDistance Function: Compares distances obtained from the left and right scans to decide the optimal direction for obstacle avoidance.

Operational Flow

1. Initialization:
 - Set up pin modes for all sensors and motors
 - Initialize serial communication for debugging
 - Set initial motor speeds using PWM
 - Position the servo motor to a default angle
2. Main Loop:
 - Continuously read values from IR sensors and the ultrasonic sensor
 - Determine movement based on IR sensor readings
 - Move forward, turn left/right, or stop
 - If both IR sensors detect black (indicating a possible obstacle), use the ultrasonic sensor to measure the distance ahead
 - If an obstacle is detected within a predefined distance
 - Use the servo to scan left and right
 - Compare distances and decide the best direction to avoid the obstacle
 - Execute the turn and continue moving forward
3. Movement Tracking:
 - After each movement command, the robot stops briefly to ensure stability and accurate tracking

Circuit Connections Overview

1. Motor Driver (L298N) to Arduino

- enA (Motor A Speed Control) → Arduino Digital Pin 10

- in1 (Motor A Input 1) → Arduino Digital Pin 9
- in2 (Motor A Input 2) → Arduino Digital Pin 8
- in3 (Motor B Input 1) → Arduino Digital Pin 7
- in4 (Motor B Input 2) → Arduino Digital Pin 6
- enB (Motor B Speed Control) → Arduino Digital Pin 5

Motor Connections (from L298N to Motors)

- Motor A (Left Motor) → Connected to L298N OUT1 and OUT2
- Motor B (Right Motor) → Connected to L298N OUT3 and OUT4

2. IR Sensors

- Left IR Sensor Output → Arduino Analog Pin A0
- Right IR Sensor Output → Arduino Analog Pin A1
- VCC of both IR Sensors → 5V from Arduino
- GND of both IR Sensors → GND on Arduino

3. Ultrasonic Sensor (HC-SR04)

- Echo Pin → Arduino Analog Pin A2
- Trigger Pin → Arduino Analog Pin A3
- VCC → 5V on Arduino
- GND → GND on Arduino

4. Servo Motor (SG90)

- Signal Pin → Arduino Analog Pin A5
- VCC → 5V on Arduino
- GND → GND on Arduino

5. Power Supply

- L298N Module Power Input (12V) → Battery Pack (7.4V or 12V depending on motor specs)
- GND of Battery Pack → Common GND with Arduino
- 5V Output from L298N (if jumper is removed) → Can be used to power Arduino via Vin

Required Apparatus:**Table 6.1: Table for Required Apparatus**

Sl. No.	Components	Specification	Quantity
1.	Arduino Uno	• ATmega328P • Operating Voltage: 5 V • Input Voltage: 7–12 V • Input Voltage: 6–20 V • Digital I/O Pins: 14 • PWM Digital I/O Pins: 6 (Pins 3, 5, 6, 9, 10, 11) • Analog Input Pins: 6 (A0 to A5) • DC Current per I/O Pin: 20 mA • DC Current for 3.3 V Pin: 50 mA • Flash Memory: 32 KB (ATmega328P) of which 0.5 KB used by bootloader • SRAM: 2 KB (ATmega328P) • EEPROM: 1 KB (ATmega328P) • Clock Speed: 16 MHz • USB Connection: Type B • LED Built-in: Pin 13	1
2.	L298N Motor Driver	• Driver Model: L298N 2A • Driver Chip: Double H Bridge L298N • Motor Supply Voltage (Maximum): 46V • Motor Supply Current (Maximum): 2A • Logic Voltage: 5V • Driver Voltage: 5-35V • Driver Current: 2A • Logical Current: 0-36mA • Maximum Power (W): 25W • Current Sense for each motor • Heatsink for better performance	1

		Power-On LED indicator	
3.	Gear Motors	- Operating Voltage: 6V - Shaft Length: 8.5mm - Shaft diameter: 5.5mm - Rated Speed: 100-150RPM - No Load Current: 40-180mA - Rated Torque: 0.35 Kg-cm - Dimensions(L×W×H) in mm: 65×23×27	2
4.	Wheel	-	2
5.	Battery	9V	1
6.	Jumper Wires	-	-
7.	IDE Software	Arduino IDE	-
8.	Breadboard	-	1
9.	Robot Chassis	-	1
10.	Glue Gun	-	1
11.	Servo Motor	Micro SG90 180 Degree Rotation	1
12.	Ultrasonic Sonar Sensor	HCSR-04	1

Circuit Diagram:

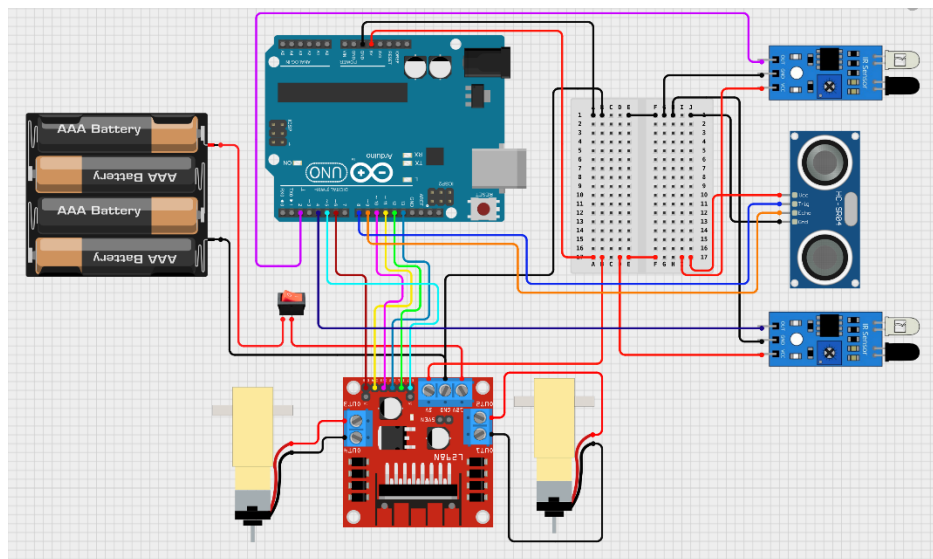


Fig.06.1: Circuit Diagram of the Line Follower Obstacle Avoiding Robot

Experimental Setup:

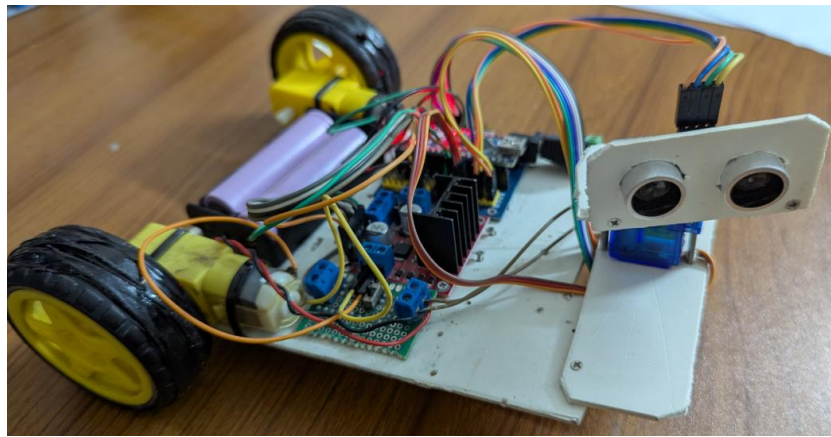


Fig.06.2: Experimental Setup of the Line Follower Obstacle Avoiding Robot

Code:

<pre>#define enA 10 // Enable1 - L298 Pin enA #define in1 9 // Motor1 - L298 Pin in1 #define in2 8 // Motor1 - L298 Pin in2 #define in3 7 // Motor2 - L298 Pin in3 #define in4 6 // Motor2 - L298 Pin in4 #define enB 5 // Enable2 - L298 Pin enB #define L_S A0 // Left IR Sensor #define R_S A1 // Right IR Sensor #define echo A2 // Ultrasonic Echo pin #define trigger A3 // Ultrasonic Trigger pin #define servo A5 // Servo motor control pin int Set = 15; // Distance threshold int distance_L, distance_F, distance_R; void setup() { Serial.begin(9600); pinMode(R_S, INPUT); pinMode(L_S, INPUT); pinMode(echo, INPUT); pinMode(trigger, OUTPUT); pinMode(enA, OUTPUT); pinMode(in1, OUTPUT); pinMode(in2, OUTPUT); pinMode(in3, OUTPUT);</pre>	<pre>void compareDistance() { if (distance_L > distance_R) { turnLeft(); delay(500); forword(); delay(600); turnRight(); delay(500); forword(); delay(600); turnRight(); delay(400); } else { turnRight(); delay(500); forword(); delay(600); turnLeft(); delay(500); forword(); delay(600); turnLeft(); delay(400); } } void Check_side() { Stop(); delay(100); for (int angle = 70; angle <= 140; angle += 5) { servoPulse(servo, angle);</pre>
--	---

<pre> pinMode(in4, OUTPUT); pinMode(enB, OUTPUT); pinMode(servo, OUTPUT); analogWrite(enA, 200); // Set speed for Motor A analogWrite(enB, 200); // Set speed for Motor B // Initialize servo scan for (int angle = 70; angle <= 140; angle += 5) { servoPulse(servo, angle); } for (int angle = 140; angle >= 0; angle -= 5) { servoPulse(servo, angle); } for (int angle = 0; angle <= 70; angle += 5) { servoPulse(servo, angle); } distance_F = Ultrasonic_read(); delay(500); } void loop() { distance_F = Ultrasonic_read(); Serial.print("D F = "); Serial.println(distance_F); if ((digitalRead(R_S) == 0) && (digitalRead(L_S) == 0)) { if (distance_F > Set) { forword(); } else { Check_side(); } } else if ((digitalRead(R_S) == 1) && (digitalRead(L_S) == 0)) { turnRight(); } else if ((digitalRead(R_S) == 0) && (digitalRead(L_S) == 1)) { turnLeft(); } delay(10); } void servoPulse(int pin, int angle) { int pwm = (angle * 11) + 500; digitalWrite(pin, HIGH); delayMicroseconds(pwm); digitalWrite(pin, LOW); delay(50); } </pre>	<pre> } delay(300); distance_R = Ultrasonic_read(); Serial.print("D R = "); Serial.println(distance_R); delay(100); for (int angle = 140; angle >= 0; angle -= 5) { servoPulse(servo, angle); } delay(500); distance_L = Ultrasonic_read(); Serial.print("D L = "); Serial.println(distance_L); delay(100); for (int angle = 0; angle <= 70; angle += 5) { servoPulse(servo, angle); } delay(300); compareDistance(); } void forword() { digitalWrite(in1, LOW); digitalWrite(in2, HIGH); digitalWrite(in3, HIGH); digitalWrite(in4, LOW); } void backward() { digitalWrite(in1, HIGH); digitalWrite(in2, LOW); digitalWrite(in3, LOW); digitalWrite(in4, HIGH); } void turnRight() { digitalWrite(in1, LOW); digitalWrite(in2, HIGH); digitalWrite(in3, LOW); digitalWrite(in4, HIGH); } void turnLeft() { digitalWrite(in1, HIGH); digitalWrite(in2, LOW); digitalWrite(in3, HIGH); digitalWrite(in4, LOW); } void Stop() { digitalWrite(in1, LOW); digitalWrite(in2, LOW); } </pre>
--	--

<pre> long Ultrasonic_read() { digitalWrite(trigger, LOW); delayMicroseconds(2); digitalWrite(trigger, HIGH); delayMicroseconds(10); long time = pulseIn(echo, HIGH); return time / 29 / 2; } </pre>	<pre> digitalWrite(in3, LOW); digitalWrite(in4, LOW); } </pre>
--	--

Experimental Procedure:

- The Arduino UNO board was connected to the computer using a USB cable.
- The necessary components, including two DC motors, IR sensors, ultrasonic sensor, servo motor, L298N motor driver module, and jumper wires, were collected.
- The left and right IR sensors were connected to analog pins A0 and A1 of the Arduino, respectively.
- The ultrasonic sensor's trigger and echo pins were connected to A3 and A2 of the Arduino, respectively.
- The servo motor was connected to analog pin A5 of the Arduino.
- The L298N motor driver module was interfaced with the Arduino using digital pins 5, 6, 7, 8, 9, and 10.
- The left motor was connected to the L298N OUT1 and OUT2 terminals, and the right motor was connected to OUT3 and OUT4.
- A suitable power source (battery) was connected to the motor driver module, and a common ground was established with the Arduino.
- The provided Arduino code was uploaded to the board using the Arduino IDE.
- The robot was placed on a surface containing a black line over a white background.
- The functioning of IR sensors was observed to ensure correct line detection.
- The ultrasonic sensor was tested by placing obstacles in the robot's path to verify obstacle detection and distance measurement.
- The servo motor was observed for correct rotation and scanning when an obstacle was detected.
- The robot's movement was monitored to confirm correct decision-making and navigation based on sensor input.
- Adjustments were made to the threshold distance and motor speeds in the code if needed for smooth operation.
- The entire setup was tested in different scenarios to validate the robot's line following and obstacle avoidance capabilities.

Hands-on Robotics Tasks for Mastery of Navigation Feedback and Movement Logging using Arduino

Task No.: 01

Task Name: Directional LED Feedback Implementation for Line Following Robot (LFR)

Objective:

- To enhance the robot's line-following behavior with visual directional feedback.
- To activate a red LED when the robot turns right.
- To activate a green LED when the robot turns left.
- To ensure both LEDs remain off when the robot moves straight.
- To integrate LED indicators with existing motor control logic using Arduino.
- To test and validate the directional LED response based on sensor inputs and robot movement.
- To improve debugging and user interaction by visually indicating the robot's directional decisions.

Code:

<pre>#define enA 10 #define in1 9 #define in2 8 #define in3 7 #define in4 6 #define enB 5 #define L_S A0 #define R_S A1 #define echo A2 #define trigger A3 #define servo A5 #define redLED 3 // Red LED for right turn #define greenLED 4 // Green LED for left turn int Set = 15; int distance_L, distance_F, distance_R; void setup() { Serial.begin(9600); pinMode(R_S, INPUT); pinMode(L_S, INPUT); pinMode(echo, INPUT); pinMode(trigger, OUTPUT); pinMode(enA, OUTPUT); pinMode(in1, OUTPUT); pinMode(in2, OUTPUT); pinMode(in3, OUTPUT); pinMode(in4, OUTPUT); pinMode(enB, OUTPUT); pinMode(servo, OUTPUT); pinMode(redLED, OUTPUT);</pre>	<pre>void compareDistance() { if (distance_L > distance_R) { turnLeft(); delay(500); forward(); delay(600); turnRight(); delay(500); forward(); delay(600); turnRight(); delay(400); } else { turnRight(); delay(500); forward(); delay(600); turnLeft(); delay(500); forward(); delay(600); turnLeft(); delay(400); } } void Check_side() { Stop(); delay(100);</pre>
--	--

<pre> pinMode(greenLED, OUTPUT); analogWrite(enA, 200); analogWrite(enB, 200); digitalWrite(redLED, LOW); digitalWrite(greenLED, LOW); for (int angle = 70; angle <= 140; angle += 5) { servoPulse(servo, angle); } for (int angle = 140; angle >= 0; angle -= 5) { servoPulse(servo, angle); } for (int angle = 0; angle <= 70; angle += 5) { servoPulse(servo, angle); } distance_F = Ultrasonic_read(); delay(500); } void loop() { distance_F = Ultrasonic_read(); Serial.print("D F = "); Serial.println(distance_F); if ((digitalRead(R_S) == 0) && (digitalRead(L_S) == 0)) { if (distance_F > Set) { forward(); digitalWrite(redLED, LOW); digitalWrite(greenLED, LOW); } else { Check_side(); } } else if ((digitalRead(R_S) == 1) && (digitalRead(L_S) == 0)) { turnRight(); digitalWrite(redLED, HIGH); digitalWrite(greenLED, LOW); } else if ((digitalRead(R_S) == 0) && (digitalRead(L_S) == 1)) { turnLeft(); digitalWrite(redLED, LOW); digitalWrite(greenLED, HIGH); } delay(10); } void servoPulse(int pin, int angle) { int pwm = (angle * 11) + 500; digitalWrite(pin, HIGH); delayMicroseconds(pwm); digitalWrite(pin, LOW); delay(50); </pre>	<pre> for (int angle = 70; angle <= 140; angle += 5) { servoPulse(servo, angle); } delay(300); distance_R = Ultrasonic_read(); Serial.print("D R = "); Serial.println(distance_R); delay(100); for (int angle = 140; angle >= 0; angle -= 5) { servoPulse(servo, angle); } delay(500); distance_L = Ultrasonic_read(); Serial.print("D L = "); Serial.println(distance_L); delay(100); for (int angle = 0; angle <= 70; angle += 5) { servoPulse(servo, angle); } delay(300); compareDistance(); } void forward() { digitalWrite(in1, LOW); digitalWrite(in2, HIGH); digitalWrite(in3, HIGH); digitalWrite(in4, LOW); } void backward() { digitalWrite(in1, HIGH); digitalWrite(in2, LOW); digitalWrite(in3, LOW); digitalWrite(in4, HIGH); } void turnRight() { digitalWrite(in1, LOW); digitalWrite(in2, HIGH); digitalWrite(in3, LOW); digitalWrite(in4, HIGH); } void turnLeft() { digitalWrite(in1, HIGH); digitalWrite(in2, LOW); digitalWrite(in3, HIGH); digitalWrite(in4, LOW); } </pre>
---	---

<pre> } long Ultrasonic_read() { digitalWrite(trigger, LOW); delayMicroseconds(2); digitalWrite(trigger, HIGH); delayMicroseconds(10); long time = pulseIn(echo, HIGH); return time / 29 / 2; } </pre>	<pre> void Stop() { digitalWrite(in1, LOW); digitalWrite(in2, LOW); digitalWrite(in3, LOW); digitalWrite(in4, LOW); } </pre>
--	--

Task No.: 02

Task Name: Movement Direction Logging into EEPROM for Line Following Robot (LFR)

Objective:

- To implement real-time logging of the robot's movement direction using EEPROM.
- To store character 'F' in EEPROM when the robot moves forward.
- To store character 'L' in EEPROM when the robot turns left.
- To store character 'R' in EEPROM when the robot turns right.
- To write each movement character at a unique and sequential EEPROM address.
- To ensure persistence of movement logs even after power loss or system reset.
- To enable future retrieval and analysis of the robot's navigation path.

Code:

<pre> #include <EEPROM.h> #define enA 10 #define in1 9 #define in2 8 #define in3 7 #define in4 6 #define enB 5 #define L_S A0 #define R_S A1 #define echo A2 #define trigger A3 #define servo A5 int Set = 15; int distance_L, distance_F, distance_R; int eepromAddress = 0; // EEPROM address pointer void setup() { Serial.begin(9600); </pre>	<pre> void compareDistance() { if (distance_L > distance_R) { turnLeft(); logMovement('L'); delay(500); forward(); logMovement('F'); delay(600); turnRight(); logMovement('R'); delay(500); forward(); logMovement('F'); delay(600); turnRight(); logMovement('R'); delay(400); } else { turnRight(); logMovement('R'); delay(500); forward(); logMovement('F'); delay(600); turnLeft(); logMovement('L'); </pre>
---	--

<pre> pinMode(R_S, INPUT); pinMode(L_S, INPUT); pinMode(echo, INPUT); pinMode(trigger, OUTPUT); pinMode(enA, OUTPUT); pinMode(in1, OUTPUT); pinMode(in2, OUTPUT); pinMode(in3, OUTPUT); pinMode(in4, OUTPUT); pinMode(enB, OUTPUT); pinMode(servo, OUTPUT); analogWrite(enA, 200); analogWrite(enB, 200); for (int angle = 70; angle <= 140; angle += 5) { servoPulse(servo, angle); } for (int angle = 140; angle >= 0; angle -= 5) { servoPulse(servo, angle); } for (int angle = 0; angle <= 70; angle += 5) { servoPulse(servo, angle); } distance_F = Ultrasonic_read(); delay(500); } void loop() { distance_F = Ultrasonic_read(); Serial.print("D F = "); Serial.println(distance_F); if ((digitalRead(R_S) == 0) && (digitalRead(L_S) == 0)) { if (distance_F > Set) { forward(); logMovement('F'); } else { Check_side(); } } else if ((digitalRead(R_S) == 1) && (digitalRead(L_S) == 0)) { turnRight(); logMovement('R'); } else if ((digitalRead(R_S) == 0) && (digitalRead(L_S) == 1)) { turnLeft(); logMovement('L'); } delay(10); </pre>	<pre> delay(500); forward(); logMovement('F'); delay(600); turnLeft(); logMovement('L'); delay(400); } } void Check_side() { Stop(); delay(100); for (int angle = 70; angle <= 140; angle += 5) { servoPulse(servo, angle); } delay(300); distance_R = Ultrasonic_read(); Serial.print("D R = "); Serial.println(distance_R); delay(100); for (int angle = 140; angle >= 0; angle -= 5) { servoPulse(servo, angle); } delay(500); distance_L = Ultrasonic_read(); Serial.print("D L = "); Serial.println(distance_L); delay(100); for (int angle = 0; angle <= 70; angle += 5) { servoPulse(servo, angle); } delay(300); compareDistance(); } void forward() { digitalWrite(in1, LOW); digitalWrite(in2, HIGH); digitalWrite(in3, HIGH); digitalWrite(in4, LOW); } void backward() { digitalWrite(in1, HIGH); digitalWrite(in2, LOW); digitalWrite(in3, LOW); digitalWrite(in4, HIGH); } </pre>
---	---

<pre> } void logMovement(char direction) { if (eepromAddress < EEPROM.length()) { EEPROM.write(eepromAddress, direction); Serial.print("Logged: "); Serial.print(direction); Serial.print(" at address "); Serial.println(eepromAddress); eepromAddress++; } else { Serial.println("EEPROM full. Logging stopped."); } } void servoPulse(int pin, int angle) { int pwm = (angle * 11) + 500; digitalWrite(pin, HIGH); delayMicroseconds(pwm); digitalWrite(pin, LOW); delay(50); } long Ultrasonic_read() { digitalWrite(trigger, LOW); delayMicroseconds(2); digitalWrite(trigger, HIGH); delayMicroseconds(10); long time = pulseIn(echo, HIGH); return time / 29 / 2; } </pre>	<pre> void turnRight() { digitalWrite(in1, LOW); digitalWrite(in2, HIGH); digitalWrite(in3, LOW); digitalWrite(in4, HIGH); } void turnLeft() { digitalWrite(in1, HIGH); digitalWrite(in2, LOW); digitalWrite(in3, HIGH); digitalWrite(in4, LOW); } void Stop() { digitalWrite(in1, LOW); digitalWrite(in2, LOW); digitalWrite(in3, LOW); digitalWrite(in4, LOW); } </pre>
---	---

Discussions:

The experiment was not successful at first try. After several attempts, the robot almost worked but faced difficulties.

Justification of CO2 and PO(j):

The directional LED feedback visually indicates real-time decisions—red for right turns and green for left—making the robot's behavior easy to understand. Besides, the EEPROM-based logging records movement history, enabling users to analyze patterns and debug logic. Together, these features increase system transparency and effectively communicate functionality and logic to a wide audience.

Conclusion:

In conclusion, the project successfully demonstrates autonomous navigation using sensor input, motor control, and decision-making logic.

Reference: <https://www.instructables.com/Line-Following-Robot-With-Obstacle-Avoidance/>